

NANYANG TECHNOLOGICAL UNIVERSITY

SEMESTER 1 EXAMINATION 2020-2021

CE2002/CZ2002 – OBJECT-ORIENTED DESIGN & PROGRAMMING

Nov/Dec 2020

Time Allowed: 2 hours

INSTRUCTIONS

1. This paper contains 4 questions and comprises 8 pages.
2. Answer **ALL** questions.
3. This is a closed-book examination.
4. All questions carry equal marks.
5. APPENDIX A shows the Sequence Diagram referenced by Question 3(b).
6. APPENDIX B shows the Class Diagram referenced by Question 4(a).

-
1. (a) (i) Explain what is ***inheritance*** in the object-oriented context. Besides inheritance, list the other **THREE** main features of object-oriented model.
(5 marks)
 - (ii) C++ supports multiple inheritance. Explain what problems will arise because of multiple inheritance. Elaborate how Java implements a similar idea in its own way, with an example of Java code.
(4 marks)
 - (b) What are the **TWO** benefits of ***polymorphism*** in object-oriented programming? Explain why polymorphism has these two benefits.
(6 marks)

Note: Question No. 1 continues on Page 2

- (c) Write Java code to demonstrate how to define a *constant*. And, list the **TWO** advantages of using constants in programming.

(6 marks)

- (d) Write Java code to demonstrate how a *mutator* method helps to maintain data consistency.

(4 marks)

2. Study the partial UML Class Diagram in Figure Q2.

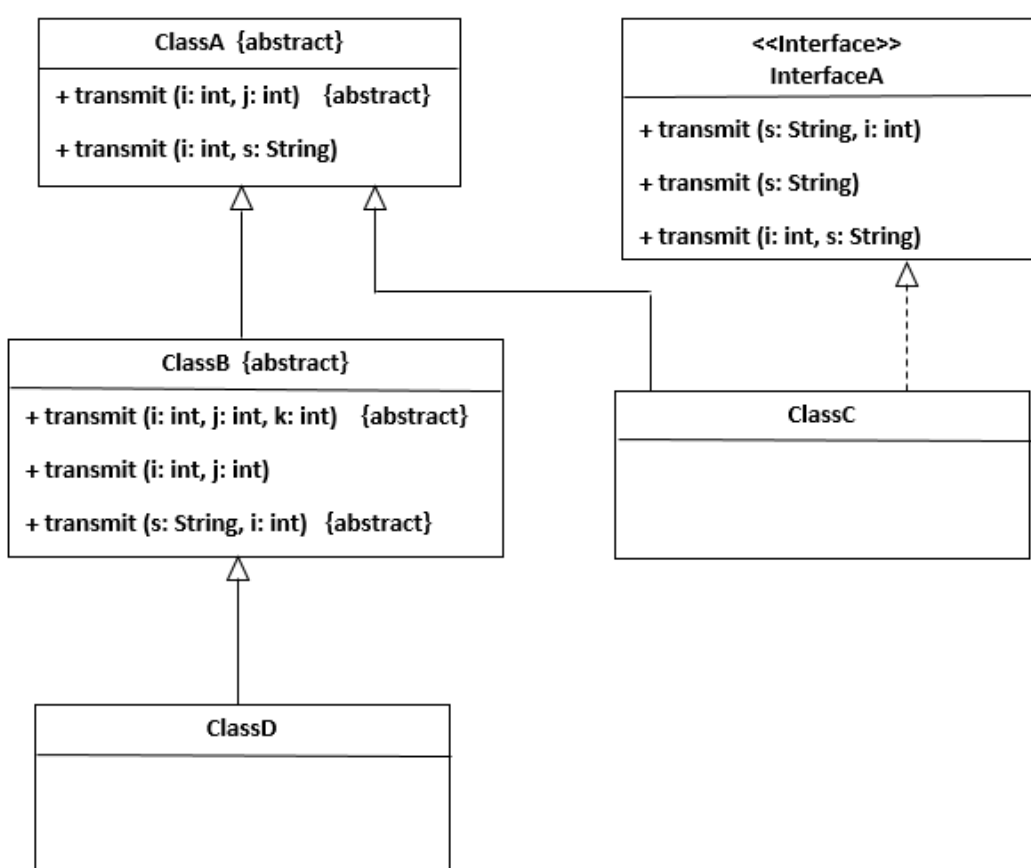


Figure Q2

- (a) ClassC and ClassD are TWO concrete classes. Which methods have to be implemented by each of these TWO classes?

(10 marks)

Note: Question No. 2 continues on Page 3

- (b) Assuming all necessary methods are implemented in `ClassC` and `ClassD`, EXPLAIN the outcome, the class's method called or used (if any) and the type of casting for **each line** of the following codes during **compile-time** and **runtime**.

(i) `ClassA a = new ClassA() ;`
`a.transmit(1, "2") ;`
 (2 marks)

(ii) `ClassB b = new ClassD() ;`
`b.transmit("2", 1) ;`
 (2 marks)

(iii) `InterfaceA a = new ClassD() ;`
`a.transmit(1, 2) ;`
 (2 marks)

(iv) `ClassA a = new ClassD() ;`
`a.transmit(1, 2) ;`
`ClassB b = a;`
`b.transmit(1, "2") ;`
 (3 marks)

(v) `ClassB b = new ClassD() ;`
`b.transmit(1, 2) ;`
`ClassA a = b;`
`a.transmit("1", 2) ;`
 (3 marks)

(vi) `InterfaceA a = new ClassC() ;`
`a.transmit("1", 2) ;`
`ClassA b = a;`
`b.transmit(1, 2) ;`
 (3 marks)

3. (a) Study the following description of a Work Tracking Application (WTA):

WTA is a system that maintains a centralized repository of all projects and tasks assigned to staff in an organisation. It contains the project and task details, including the tracking of issues. WTA allows the easy access of relevant information on the staff and projects as well as tracking a project's progress and staff's utilisation rate. A staff can play different roles in projects.

Note: Question No. 3 continues on Page 4

A project has an identification number, a description, a project manager, the project awarded price and the customer. A project which is more than 1 million dollar will require also a project director. A project is broken down into multiple tasks which are assigned to the same or different staff. Each task will have a project leader, its team members, a task identification, task description and an expected date of completion. Project issues are tracked for every task in the project by assigning an identification number and providing details like title, detailed description, status, reported date and last updated date. A project is funded externally by a customer which the system will store the company name, company registration number, contact person and number.

WTA also stores the staff details like identification number, name, assigned projects and tasks, skillsets and man-hour rate. Staff will be assigned to ONE to TWO projects but multiple tasks within the same project depending on their skillsets. Skillsets like certification name, the level, awarded date and date of expiry are stored. Every month, each staff is required to clock his/her monthly utilisation details by recording the time spent on the assigned projects and tasks. Details such as the month and year, period of work (from date and to date) and type of work performed. The type of work can be administrative, project, on leave, training and mentoring.

You are tasked to identify the **entity** classes needed to build the application based on the description above.

Show your design in a Class Diagram. Your Class Diagram should show clearly the relationships between classes, enumeration, relevant attributes (at least TWO), logical multiplicities, meaningful role names, association names and constraint(s), if any. You need not show the class methods.

(14 marks)

- (b) The UML **Sequence Diagram** in Appendix A (page 7) shows the objects' interactions of a scenario flow in a particular application. Using the details depicted in the diagram, write the preliminary Java code for the `Controller` class and its methods. You may make appropriate assumptions on the method parameters, return types and return value(s) if they are not stated in the diagram.

(11 marks)

4. (a) The UML **Class Diagram** in Appendix B (page 8) shows the relationships of FOUR classes : Tutorial, Student, Instructor and TeachingAsst. TeachingAsst class inherits from both Student class and Instructor class. Study the class diagram and the details depicted carefully.

Additional details relevant to this question are provided below.

The totalTutorial attribute in both the Student class and Instructor class counts the number of tutorial registered and taught respectively. The registerTut(..) function in Student class stores the Tutorial object argument in its Tutorial array tuts attribute and increments the totalTutorial count. The teach(..) function in Instructor class stores the Tutorial object argument in its Tutorial array tuts attribute and increments the totalTutorial count. The parameters of the TeachingAsst class constructor mhours, cgpa, and r are used to initialize its own attribute minReqHours, Student class's cgpa and Instructor class's hourRate respectively. The checkConflict() function checks that there are no common Tutorial course code in both the Tutorial array in the Student class and Instructor class by using the getCourseCode() function in the Tutorial class. It will return true if there is a common course code and false if there is none.

- (i) Write the C++ code for the TeachingAsst class with all declarations and implementation in implementation file, **teachingasst.cpp**. You can assume that the Tutorial class, Student class and Instructor class are already implemented in the **studInstruct.h** header file.

(10 marks)

- (ii) Write the main function to create a TeachingAsst object, ta, and TWO Tutorial objects, t1 and t2. ta will register for t1 and teach t1 and t2. Print the result of the checkConflict function. Use the code below and continue your implementation. You can decide the data to use to instantiate the objects.

```
int main() {
    TeachAsst *ta =    // add your code
    Tutorial *t1 =     // add your code
    Tutorial *t2 =     // add your code
    // ....
}
```

Note: Question No. 4 continues on Page 6

[You should use the appropriate C++ keyword/s, pointer and reference, if necessary, to ensure the code will execute as expected with no memory leaks.]

(5 marks)

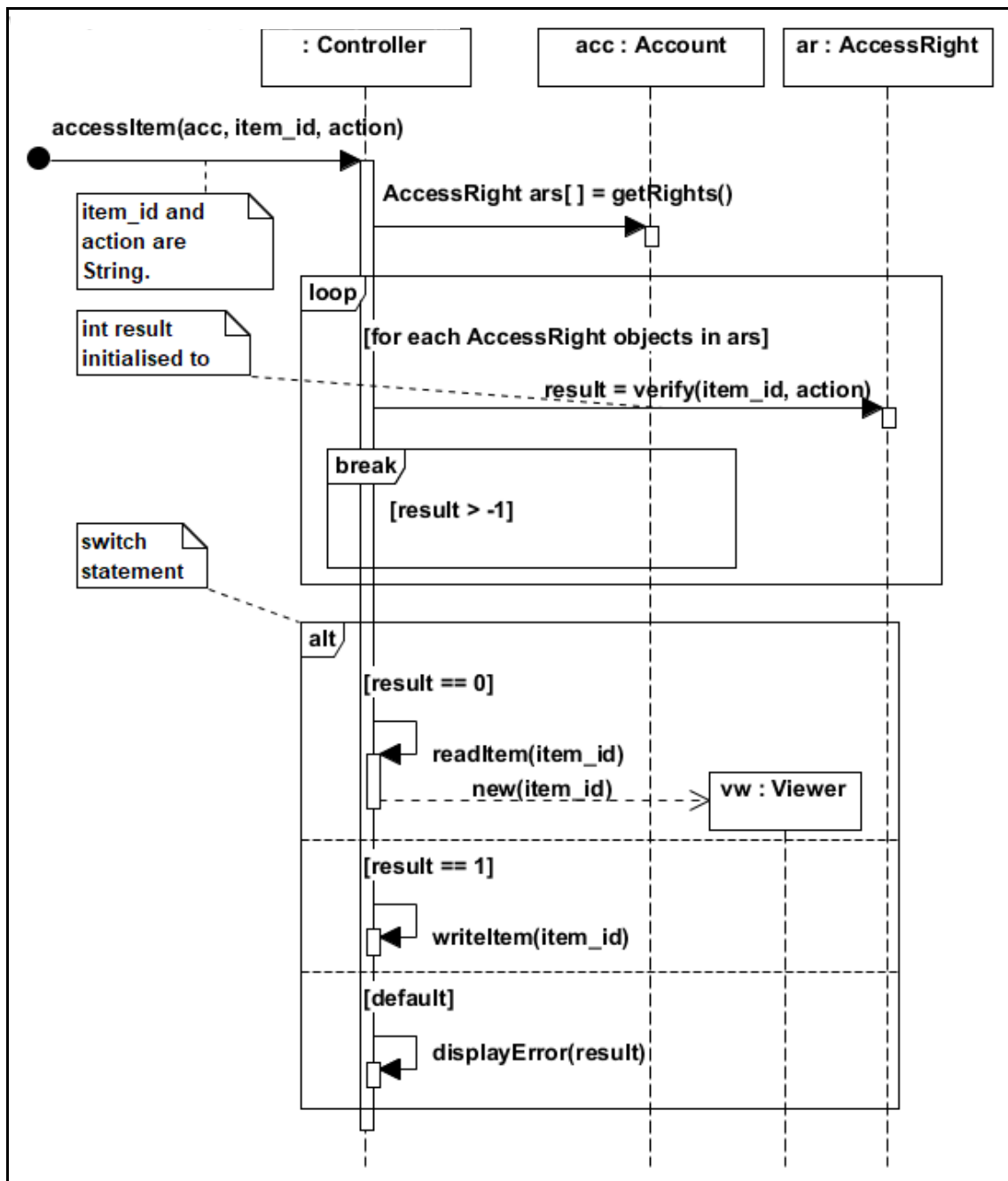
- (b) Further enhancements were suggested to the design depicted in Q4(a) to include Lecture and Laboratory sessions, in addition to the current Tutorial sessions, which Student can register for and Instructor can also teach. The THREE types of lessons have different hourly rates and minimum hour requirements. And also include a calculatePay function to calculate the instructor's total pay for teaching the THREE types of lesson.

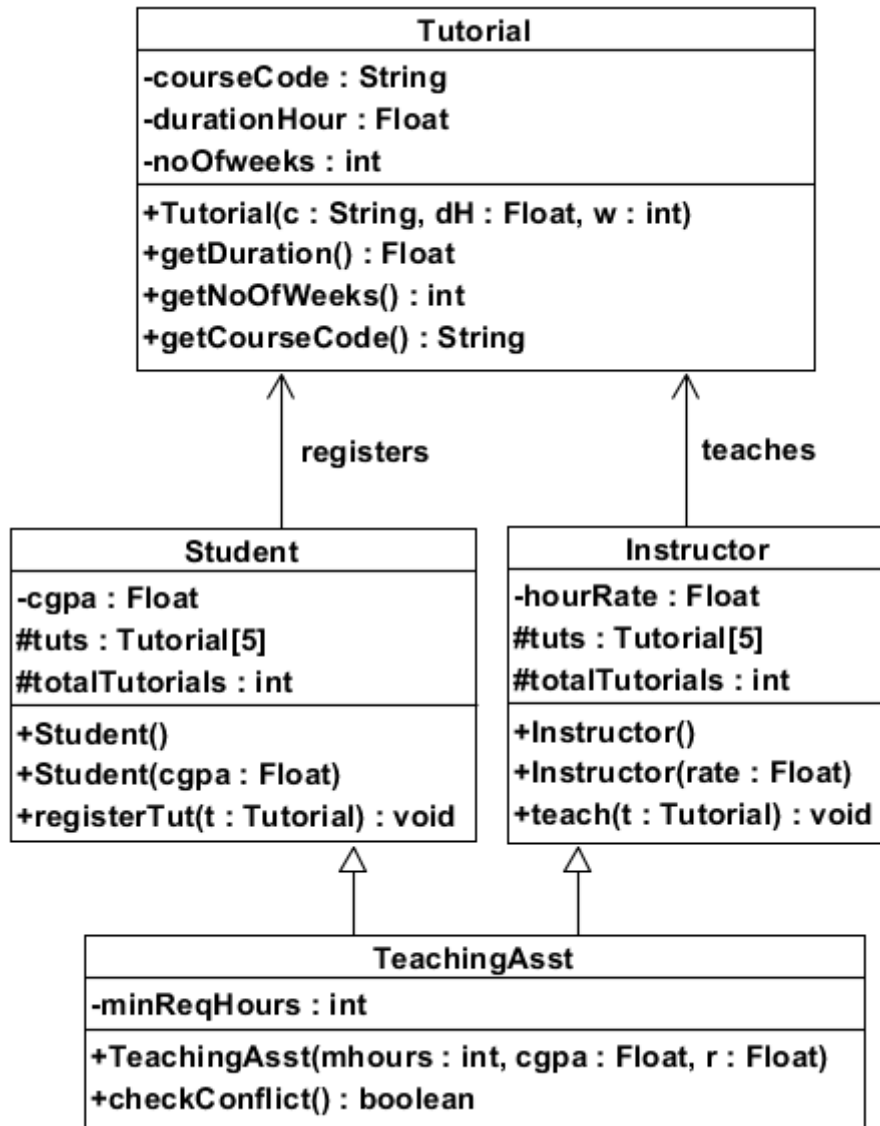
By applying the SOLID design principles, suggest and explain using a Class Diagram how you would design to incorporate the TWO suggested enhancements and minimize the impact of the change/s to the existing classes at the same time.

Use TWO of the SOLID design principles to explain how they were applied.

(10 marks)

APPENDIX A: Sequence Diagram



APPENDIX B: Class Diagram

END OF PAPER